

Практическая работа № 8

Сегментация изображений

Цель работы

Ознакомиться с основными методами сегментации изображений, научиться применять алгоритмы пороговой, кластеризационной и региональной сегментации, а также использовать методы машинного обучения для выделения объектов.

Теоретическая часть

1. Пороговая сегментация

- Простая бинаризация: пиксели делятся на «объект» и «фон» по выбранному порогу.
- Метод Отсу — автоматический выбор порога на основе минимизации внутриклассовой дисперсии.
- Адаптивная бинаризация — порог вычисляется локально для разных областей изображения.

2. Региональные методы

- **Region Growing** (рост областей) — объединение соседних пикселей на основе схожести.
- **Watershed (линии водораздела)** — интерпретация изображения как топографической карты, где границы объектов соответствуют водоразделам.

3. Кластеризационные методы

- **K-means** — жёсткое разбиение пикселей на K кластеров.
- **Fuzzy C-means (FCM)** — нечёткая кластеризация, где пиксель может частично принадлежать нескольким кластерам.
- **Mean Shift** — поиск областей высокой плотности в пространстве признаков.

4. Методы машинного обучения

- **Классические ML-методы:** SVM, Random Forest, обучаемые на пиксельных признаках.
 - **Современные DL-методы:** свёрточные нейросети (U-Net, Mask R-CNN, DeepLab).
 - Используются для семантической и instance-сегментации.
 - Требуют обучающей выборки с разметкой (ground truth masks).
-

Пример программы

1. Пороговая сегментация (метод Отсу)

```
import cv2
import matplotlib.pyplot as plt

# Загружаем изображение в градациях серого
img = cv2.imread('image.jpg', 0)

# Порог по методу Отсу
_, otsu = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

plt.subplot(1,2,1), plt.imshow(img, cmap='gray'), plt.title("Исходное")
plt.subplot(1,2,2), plt.imshow(otsu, cmap='gray'), plt.title("Отсу")
plt.show()
```

2. Watershed

```
import numpy as np

img = cv2.imread('cells.png')
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 0, 255,
cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)

# Морфологическая обработка
kernel = np.ones((3,3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)

# Фон и маркеры
sure_bg = cv2.dilate(opening, kernel, iterations=3)
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist_transform, 0.7*dist_transform.max(), 255, 0)

sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

# Маркеры
_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown==255] = 0

markers = cv2.watershed(img, markers)
img[markers == -1] = [255, 0, 0]

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Watershed сегментация")
```

```
plt.show()
```

3. Сегментация с помощью K-means

```
import numpy as np
```

```
image = cv2.imread('image.jpg')
```

```
Z = image.reshape((-1,3))
```

```
Z = np.float32(Z)
```

```
# Задаем критерий и число кластеров
```

```
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,  
10, 1.0)
```

```
K = 3
```

```
_, labels, centers = cv2.kmeans(Z, K, None, criteria, 10,  
cv2.KMEANS_RANDOM_CENTERS)
```

```
centers = np.uint8(centers)
```

```
segmented_img = centers[labels.flatten()]
```

```
segmented_img = segmented_img.reshape((image.shape))
```

```
plt.imshow(cv2.cvtColor(segmented_img, cv2.COLOR_BGR2RGB))
```

```
plt.title("K-means сегментация")
```

```
plt.show()
```

Контрольные вопросы

1. Что такое сегментация изображения?
2. В чем отличие глобальной и адаптивной бинаризации?

3. Опишите принцип работы метода Watershed.
4. Чем отличается K-means от Fuzzy C-means?
5. Какие преимущества у сверточных нейронных сетей для сегментации?